BUILDSITE

UML DESIGN DOCUMENT

Java • Spring Boot • Spring MVC • Thymeleaf • Spring Data JPA • H2

Visual UML diagrams with class boxes, attributes, methods, and relationships

1. Project Overview

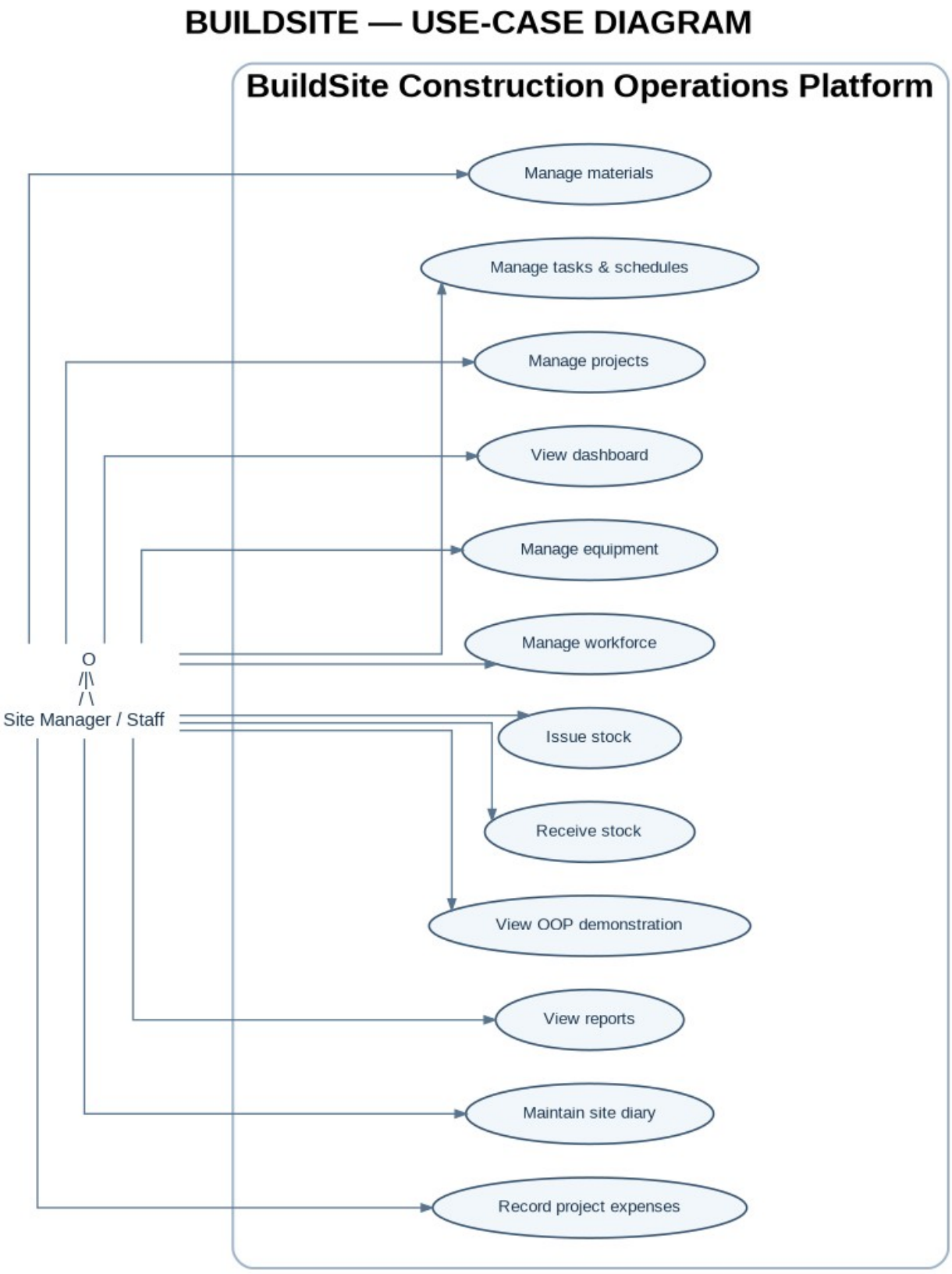
BuildSite is a server-rendered construction-site management application. The diagrams in this report represent the Java application structure and the main data relationships. The class boxes use standard UML-style compartments: class name, attributes, and operations where applicable.

Main modules included:

- Dashboard and summary metrics
- Projects and tasks / schedules
- Materials and inventory receiving / issuing
- Workforce and equipment records
- Project expenses and site diary
- Reports and an object-oriented programming demonstration

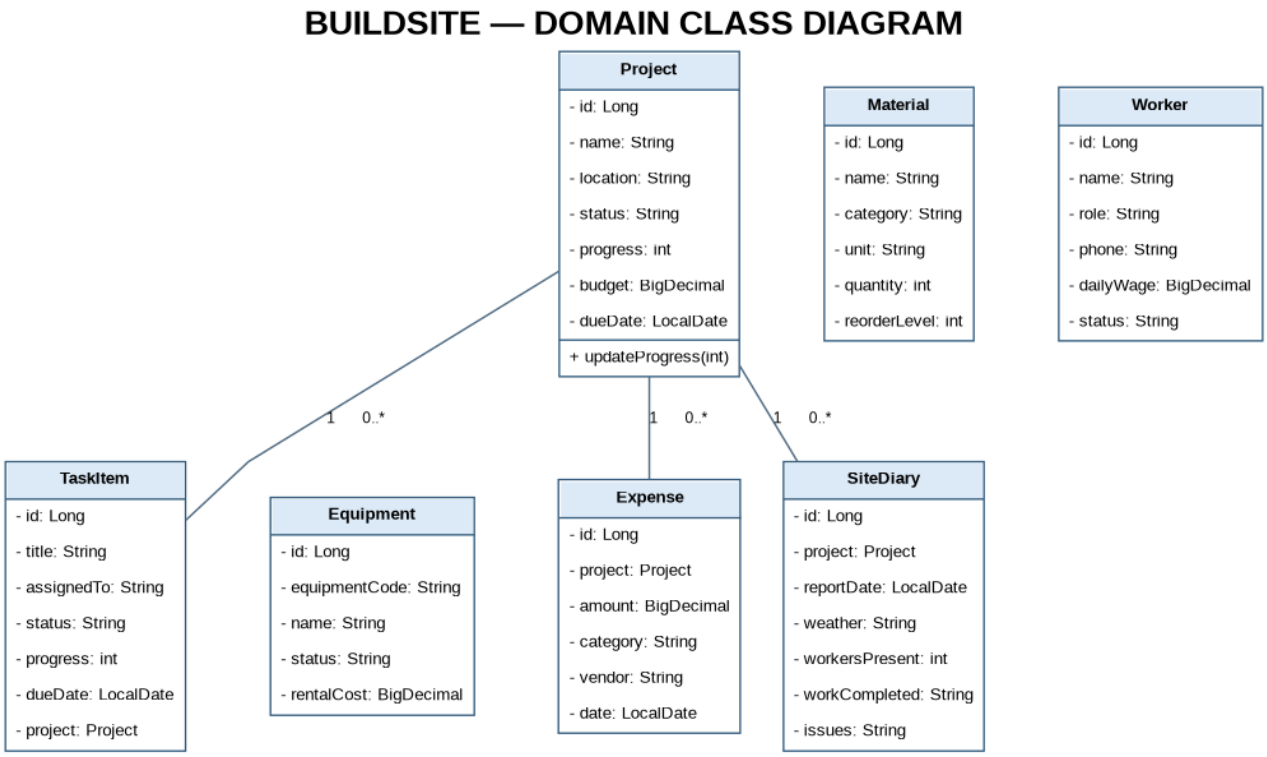
Implementation note: the current project is a development/demo application. The README indicates that production authentication/authorization, audit logging, attendance, and formal stock transaction history are not implemented. The OOP demo types are examples, not authenticated user accounts.

2. Use-Case Diagram



The actor represents a general site manager or staff member. Use cases show the functions available through the application.

3. Domain Class Diagram

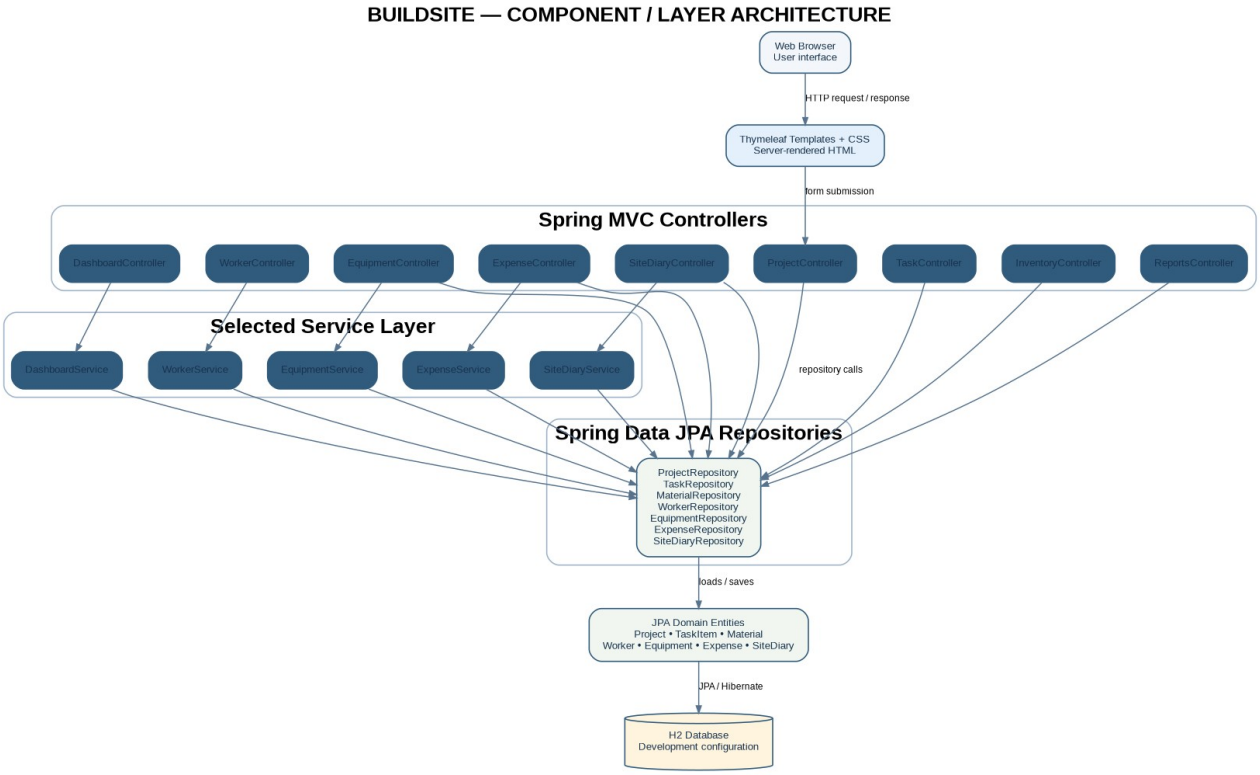


UML boxes show attributes and methods. Project has one-to-many relationships with TaskItem, Expense, and SiteDiary. Material, Worker, and Equipment are separate domain records.

Relationship summary

- Project 1 — 0..* TaskItem: a project can be associated with multiple tasks.
- Project 1 — 0..* Expense: project expenses reference the project they belong to.
- Project 1 — 0..* SiteDiary: each diary entry is associated with a project.
- Material, Worker, and Equipment are standalone records in the current domain model.

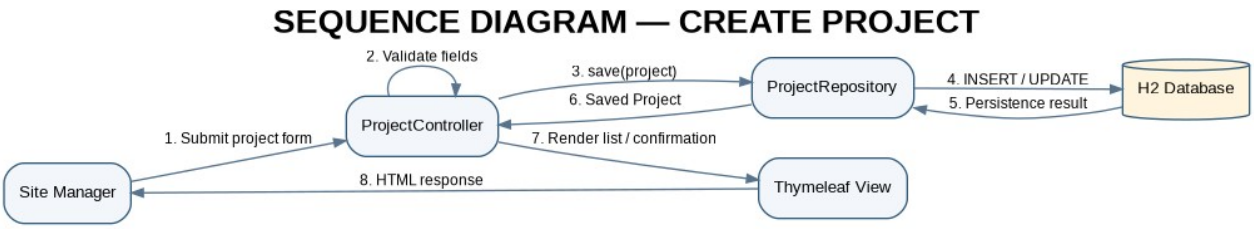
4. Component / Layer Architecture



The app follows a typical Spring MVC flow. Some controllers call repositories directly; only selected areas use dedicated service classes.

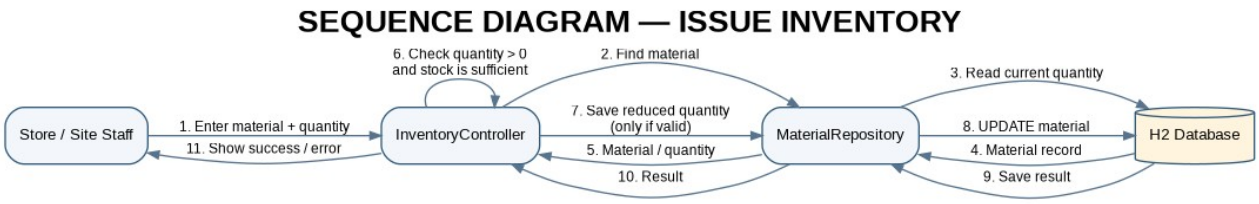
Technology path: browser → Thymeleaf templates and CSS → Spring MVC controllers → selected service classes / Spring Data JPA repositories → JPA entities → H2 database.

5. Sequence Diagram — Create Project



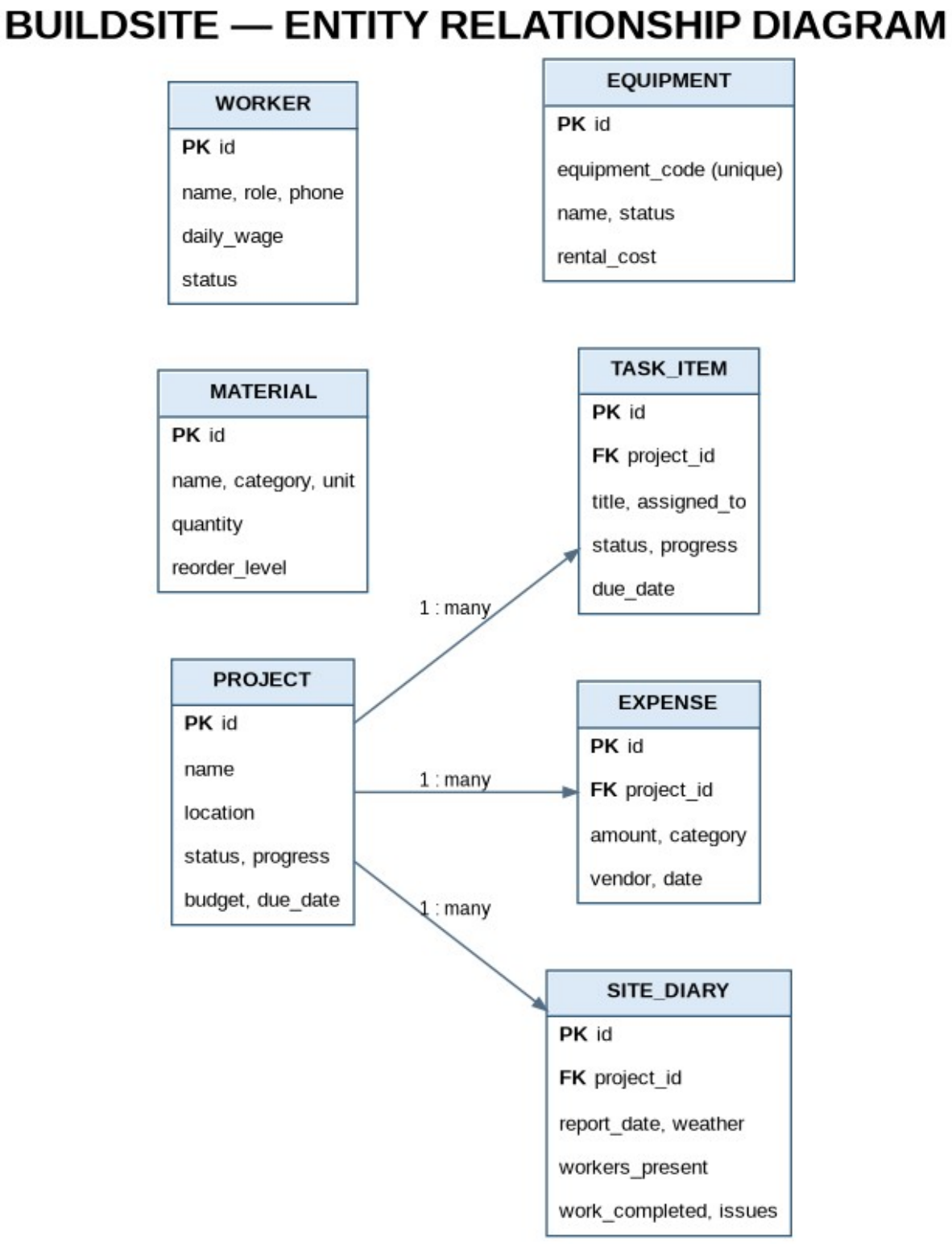
This sequence shows the typical form-submission flow from the user through the controller and repository to database persistence.

6. Sequence Diagram — Issue Inventory



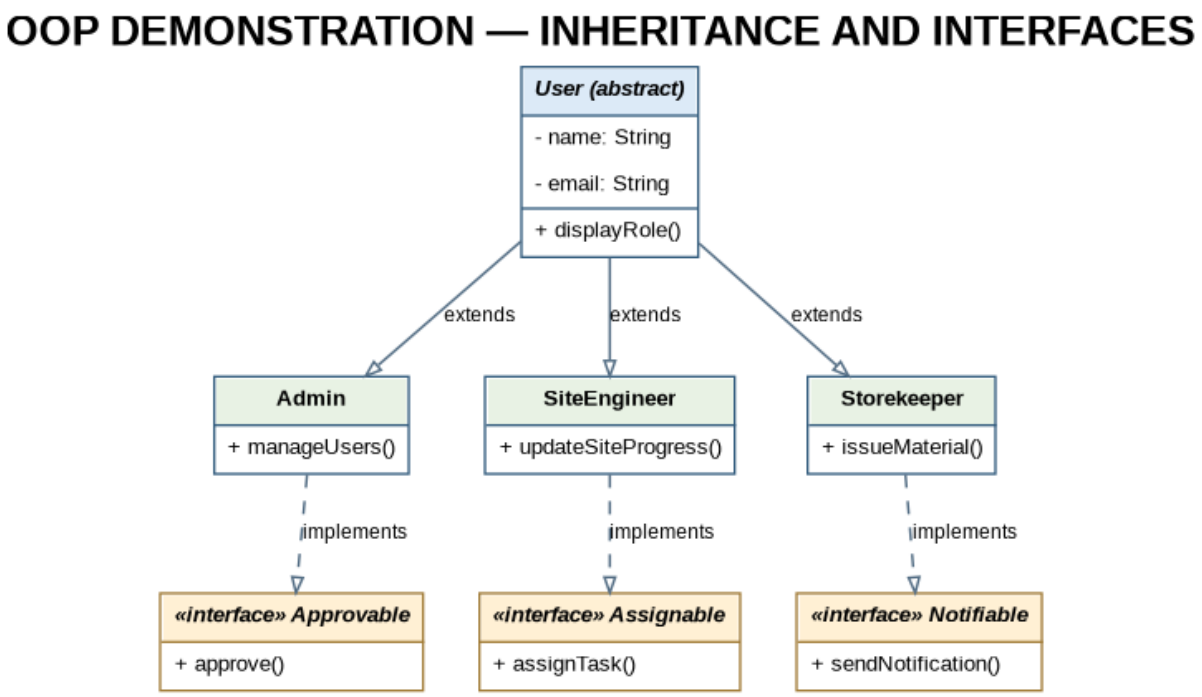
The controller should reject a non-positive issue quantity or insufficient stock, and only persist the reduced quantity when validation succeeds.

7. Entity Relationship Diagram (ERD)



PK means primary key; FK means foreign key. The three project-owned tables shown are TaskItem, Expense, and SiteDiary.

8. OOP Demonstration Class Diagram



This diagram describes the teaching/demo OOP hierarchy (abstract User, role-specific subclasses, and interfaces). Treat it as a demonstration separate from the database-backed operational entities. Exact interface implementation should be verified against the source code before describing it as production behavior.

9. Main Navigation Map

Route	Page / module
/	Dashboard
/projects	Projects
/tasks	Tasks and schedule
/inventory	Materials and inventory
/workforce	Workforce
/equipment	Equipment
/expenses	Project expenses
/site-diary	Site diary
/reports	Reports
/oop-demo	OOP demonstration

10. Important Design Constraints

- Project and task progress should remain between 0 and 100.
- Inventory quantities and reorder levels must not be negative.
- Issuing inventory must validate positive quantity and available stock.
- Equipment code should be unique; rental cost should not be negative.
- Expenses must have a valid project reference and positive amount.
- Site diary entries must be linked to a valid project.
- Do not treat the current demo app as production-secure without authentication, authorization, audit logging, and suitable deployment hardening.

11. Recommended Future Improvements

- Add UserAccount and Role entities with Spring Security.
- Add StockTransaction to record each receipt/issue, date, quantity, reason, and responsible user.
- Add Worker attendance linked to a project and date.
- Add equipment assignment and maintenance history.
- Introduce enums for status fields and database migration tooling.
- Move project, task, and inventory business rules into dedicated service classes where appropriate.

12. UML Notation Key

- A solid class box contains the class name and may include attribute and operation compartments.
- A line between classes indicates an association; multiplicity 1 to 0..* means one project can have zero or many related records.
- PK = primary key; FK = foreign key.
- A dashed line with a hollow triangle in the OOP diagram represents interface realization (implements); a solid line with a hollow triangle represents inheritance (extends).